

TP Identification - Réponses

1. Utilisation du code de calcul par éléments finis

Extraction des mesures

On utilise readMeasurement pour charger les données. Pour le cas inhomogène (fichiers measure_sb.npz et measure_sb_Pi.npz), on obtient :

- $\phi_1 = 0.1955$, $\phi_2 = -0.0666$, $\phi_3 = -0.0624$, $\phi_4 = -0.0666$
- u_m : champ de pression sur 803 noeuds, compris entre 0 et 1
- P_i : projecteur 4x803 tel que $P_i @ u_m = [P_1, P_2, P_3, P_4]$
- $P_m = P_i @ u_m$, le vecteur de mesures

Pour le cas homogène (measure_sb_H.npz), les flux sont proches mais différents :

- $\phi_1 = 0.1890$, $\phi_2 = -0.0654$, $\phi_3 = -0.0583$, $\phi_4 = -0.0653$

Fonction de calcul direct

J'ai créé une fonction calcul_direct(mesh, k1, k2, phi) qui :

- Assemble les matrices EF élémentaires (via EIMats)
- Construit K_0 avec assembleK en utilisant paramdict = {1: k1, 2: k2}
- Impose les conditions de Neumann : sur chaque buse, le flux normal est ϕ_i / L où $L = 0.02$ mm est la largeur d'une buse. Le dictionnaire Neumann est {1: ϕ_1/L , 2: ϕ_2/L , 3: ϕ_3/L , 4: ϕ_4/L }
- Impose la pression nulle au noeud 0 via DirichletScalar avec zerozero=True pour fixer la constante
- Résout $K p = f$ avec spsolve
- Applique la translation (équation 4) pour recalculer la solution sur les mesures

La translation consiste à calculer :

$$t = 1/4 * \sum (P_i @ u_m)_i - (P_i @ p)_i$$
$$p_{\text{corrigé}} = p + t$$

Comparaison avec u_m

La solution obtenue n'est pas parfaitement égale à u_m , même avec les bons paramètres $k_1=1$, $k_2=2$. La différence vient du fait que :

- Le problème direct utilise des conditions de Neumann (flux imposés) alors que la mesure u_m a été générée avec des conditions de Dirichlet (pressions imposées)
- Même avec la translation, les champs ne sont pas identiques car la translation ne corrige qu'un décalage constant, pas les éventuelles différences de forme du champ
- Les flux $\phi_1.. \phi_4$ imposés en Neumann sont des valeurs scalaires (intégrales) et on perd donc de l'information par rapport au champ complet

Conséquence sur l'identification : le résidu ne pourra jamais être nul, même avec les vrais paramètres. L'identification se fait au mieux, avec un résidu minimum non nul.

2. Identification par substitution (méthode des sensibilités)

2.1 Cas à un paramètre

J'utilise les mesures homogènes (measure_sb_H). La valeur cible est $k_{\text{ref}} = 1$.

J'implémente l'algorithme de Newton :

- Initialiser $k = k_0$ (par exemple $k_0 = 0.5$)
- Pour chaque itération :
 - Calculer p_0 avec `calcul_direct` pour le paramètre k
 - Pour $d = 1$ (une seule dimension), perturber k de ϵ : $k_{\text{tilde}} = k + \epsilon$
 - Calculer p_d avec k_{tilde}
 - $J = (p_d - p_0) / \epsilon$ (différences finies)
 - $S = P_i @ J$ (matrice de sensibilité)
 - $g = S.T @ (P_i @ p_0 - P_m)$
 - $H = S.T @ S$
 - $\Delta = \text{résoudre } H \Delta = g$
 - $k = k - \Delta$

J'ai tracé les courbes :

- Le résidu (norme de $P_i @ p - P_m$) décroît rapidement puis se stabilise
- L'erreur $|k - k_{\text{ref}}|$ converge vers une valeur proche de 0

La convergence est bonne car le problème à 1 paramètre est bien conditionné. On retrouve bien $k \sim 1$.

2.2 Cas à deux paramètres

J'utilise les mesures inhomogènes (`measure_sb`). Les valeurs cibles sont $k_{1_ref} = 1$, $k_{2_ref} = 2$.

L'algorithme est le même mais avec 2 dimensions :

- J a 2 colonnes : une pour la perturbation de k_1 , une pour k_2
- H est une matrice 2×2
- g est un vecteur de taille 2

J'ai testé plusieurs points de départ :

k_0 initial	k trouvé	Résidu final
(0.5, 1)	proche de (1, 2)	faible
(2, 1)	proche de (1, 2)	faible
(5, 5)	s'éloigne ou converge mal	élevé

Observations :

- Le résidu décroît en quelques itérations
- L'erreur en norme 2 diminue mais peut stagner si le point de départ est trop éloigné
- Le problème est mal conditionné quand k_1 et k_2 sont très différents
- La sensibilité au paramètre du disque central (k_2) est plus faible, ce qui rend son identification plus difficile

La précision est bonne quand on part de valeurs raisonnables, mais se dégrade pour des points initiaux trop éloignés à cause du mauvais conditionnement de la Hessienne.

Conclusion

Ce TP a montré la mise en oeuvre d'une méthode d'identification de paramètres par éléments finis.

Importance de la précision du modèle : Le modèle EF utilisé (maillage, conditions aux limites) influence

Fonctionnement de la régularisation : La régularisation de Tikhonov ajoute un terme $\mu/2 * \|k - k_v\|^2$ à la fonction coût. Cela pénalise les grandes déviations par rapport à une valeur a priori k_v . Mathématiquement, cela revient à ajouter $\mu * I$ à la Hessienne, ce qui améliore son conditionnement. Un grand μ force la solution vers k_v , au détriment de l'ajustement aux données. C'est un compromis biais/variance classique.

Temps de calcul et performances : La méthode de Newton (substitution) converge en peu d'itération